
LLD – DAY 7

Case Study: Document Editor (Google Docs like system)

Problem Statement

Create a **Document Editor** where user can add:

- Text
- Images

✦ And system should be **scalable**, so that in future we can add:

- Video
- New line
- Tab space
- Other document elements

👉 Follow **SOLID principles** while designing.

Approach to Solve LLD Problems

1 Top-Down Approach

- Pehle **main object** banao
- Phir **small objects**

2 Bottom-Up Approach

- Pehle **small objects**
- Phir **main object**

✦ Generally used by ~90% developers

👉 Is problem me **Bottom-Up approach** use kiya gaya hai

💡 Reason: Chhote-chhote components clear hote hain → system scalable hota hai.

Initial Design – BAD DESIGN

DocumentEditor (Bad Design)

💡 Idea

- Text aur Image dono ko **string** ki tarah store kar rahe hain

- Kyunki hume nahi pata user kis order me add karega

Text Diagram – Bad Design

```
+-----+
|      DocumentEditor      |
+-----+
| vector<string> elements  |
+-----+
| addText(string text)    |
| addImage(string path)   |
| renderDocument()        |
| saveToFile()            |
+-----+
```

✗ Problems in Bad Design

1. ✗ Breaks OCP

- New feature (video, newline) add karne ke liye → existing code modify karna padega

2. ✗ Breaks SRP

- Rendering
- Element handling
- Saving sab ek hi class me

3. ✗ Not Implemented

- LIP ✗
- DIP ✗
- ISP ✗

 Basically ek "God Class" ban gayi hai

◆ BETTER DESIGN – Proper Explanation

✦ Goal of Better Design

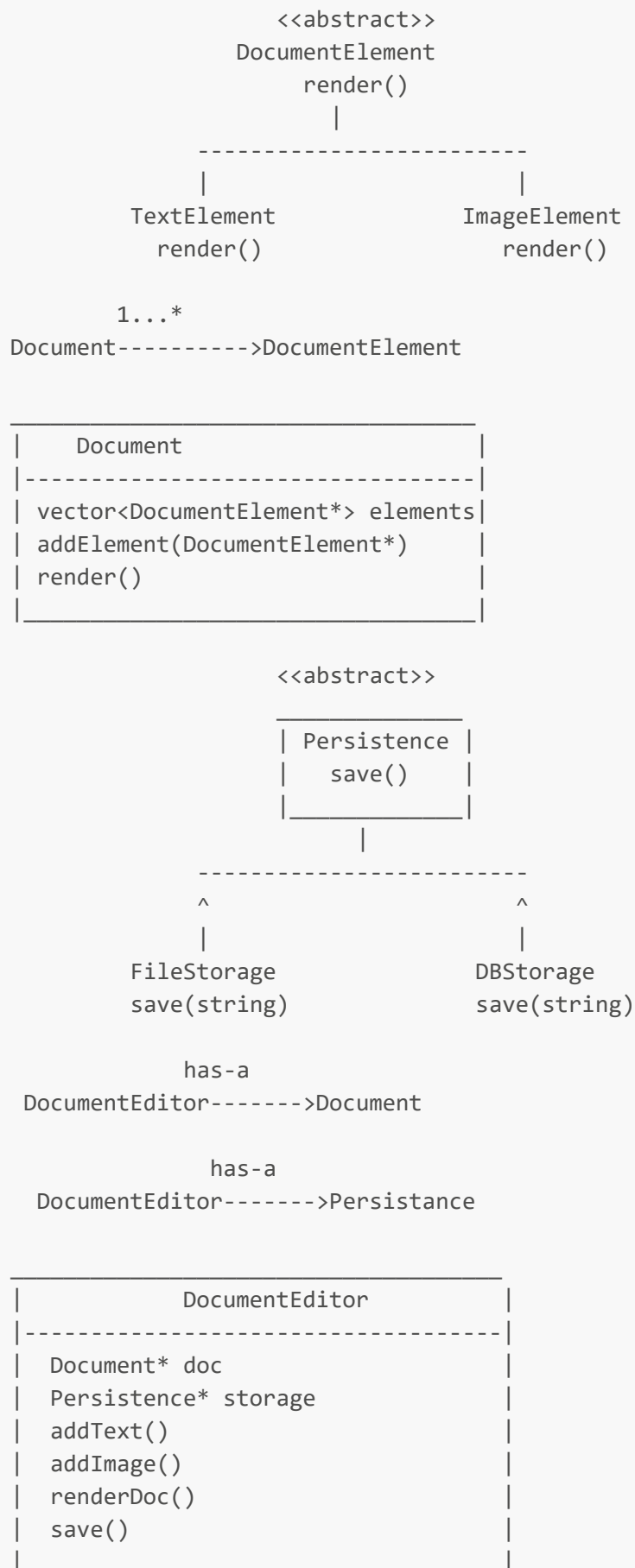
Bad design ke problems ko solve karna:

- **if-else** logic hataana
- Hard-coding kam karna
- Abstraction introduce karna

Lekin abhi **SRP fully follow nahi hota**.



TEXT UML DIAGRAM – BETTER DESIGN (Document Editor)



```
Client -----> DocumentEditor
```

🧩 1. DocumentElement (Abstraction)

```
<<abstract>>
DocumentElement
    render()
```

✅ Kya karta hai?

- Ye ek **base abstraction** hai
- Har document ka element **render()** karega

💡 Why needed?

- Agar kal **VideoElement / TableElement** add karna ho → existing code modify nahi karna padega ✅
OCP follow

🧩 2. TextElement & ImageElement

```
TextElement      ImageElement
render()         render()
```

✅ Kya karta hai?

- Har element **apna render logic khud define** karta hai
- Document ko ye jaanne ki zarurat nahi:
 - kaunsa text hai
 - kaunsa image hai

💡 Benefit

- `if (type == image)` jaisa logic remove ho gaya ✅ **Polymorphism used**

🧩 3. Document (Better design ka weak point)

```
Document
vector<DocumentElement*> elements
```

```
addElement()  
render()
```

✗ Problem yahin se start hoti hai

Document:

- elements **store bhi** karta hai
- elements ko **render bhi** karta hai

👉 **Two responsibilities**

- Data holding
- Rendering logic

✗ **SRP violation**

🧩 4. Persistence Abstraction

```
<<abstract>>  
Persistence  
save()
```

✅ Kya karta hai?

- Saving ka contract define karta hai
- DocumentEditor ko ye nahi pata:
 - file me save ho raha hai
 - DB me save ho raha hai

✅ **DIP follow**

🧩 5. FileStorage & DBStorage

FileStorage	DBStorage
save()	save()

💡 Benefit

- Storage change karne pe editor ka code change nahi hota ✅ **OCP + DIP**
-

🧩 6. DocumentEditor (Another weak point)

```
DocumentEditor
- addText()
- addImage()
- renderDoc()
- save()
```

✗ Problem

DocumentEditor:

- elements create karta hai
- rendering trigger karta hai
- persistence call karta hai

👉 **Coordinator + Creator + Controller ✗ Multiple responsibilities**

🎯 BETTER DESIGN SUMMARY

- ✓ Bad design se kaafi better
- ✓ Abstraction + Polymorphism used
- ✗ SRP fully satisfied nahi
- ✗ Document & Editor overloaded

📌 Interview line

Better design removes bad practices but may still have mixed responsibilities.

◆ OPTIMAL DESIGN – Proper Explanation

📌 Goal of Optimal Design

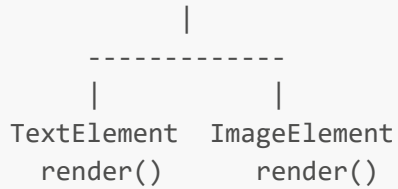
Har class = ek responsibility

Aur:

- Easily extendable
- Easily testable
- Production-grade LLD

🖋️ TEXT UML DIAGRAM – OPTIMAL DESIGN

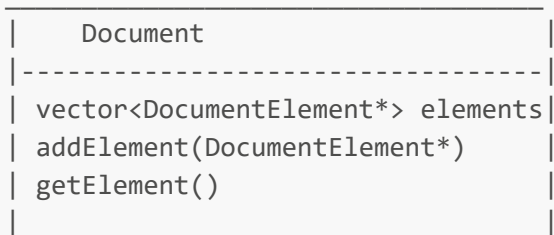
```
<<abstract>>
DocumentElement
```



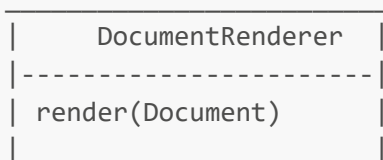
1...*

Document----->DocumentElement

<<model>>



DocumentRenderer----->Document



<<abstract>>

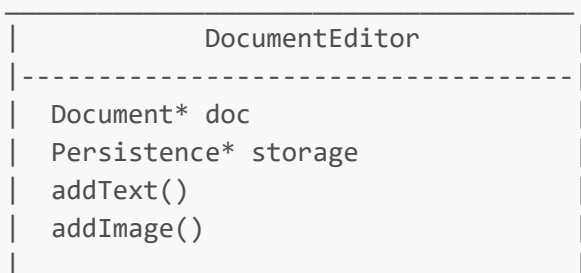
Persistence

save(string data)



DocumentEditor----->Document

DocumentEditor----->Persistence



Client -----> Persistence

Client -----> Document

```
Client -----> DocumentEditor
```

```
Client -----> Documentation
```

🧩 1. DocumentElement hierarchy (same, but clean)

```
DocumentElement
|
TextElement  ImageElement
```

✅ Same benefit

- Polymorphism
- OCP
- No `if-else`

🧩 2. Document (ONLY data holder now)

```
Document
- vector<DocumentElement*>
- addElement()
- getElement()
```

✅ Improvement

- **Rendering hata diya**
- Ab sirf:
 - structure maintain karta hai

✔ SRP satisfied

🧩 3. DocumentRenderer (New class – BIG WIN)

```
DocumentRenderer
render(Document)
```

🧠 Kya solve hua?

- Rendering logic **Document se bahar**

- Agar kal:
 - HTML render
 - PDF render
 - Markdown render

👉 New renderer add karo, document untouched

✓ **SRP + OCP**

🧩 4. Persistence (same as before, but cleaner)

```
Persistence  
save(data)
```

✓ **Benefit**

- Editor sirf abstraction jaanta hai
- Storage change → editor safe

✓ **DIP**

🧩 5. DocumentEditor (Now PURE coordinator)

```
DocumentEditor  
- uses Document  
- uses Persistence
```

✓ **Ab kya karta hai?**

- Client input leta hai
- Correct object ko delegate karta hai

✗ **Rendering logic nahi**

✗ **Storage logic nahi**

✓ **Single Responsibility**

🧩 6. Client Dependency (Correct flow)

```
Client -> DocumentEditor
```

Client directly:

- storage
- document
- renderer

pe depend nahi karta

✓ **Loose coupling**

OPTIMAL DESIGN SUMMARY

- ✓ Strict SRP
 - ✓ Fully SOLID compliant
 - ✓ Easily extensible
 - ✓ Interview-ready architecture
-

Short Summary: Better vs Optimal Design (Core Difference)

Better Design

Better design me **abstraction hoti hai**, lekin classes ko **abhi bhi doosri layers ki knowledge hoti hai**.

Tumhara exact point:

- **Document** ko pata hai **render()** ka concept (jabki rendering ka kaam **DocumentElement** / renderer ka hona chahiye)
- **DocumentEditor** ko pata hai:
 - save ho raha hai
 - **Persistence** ka **save()** method exist karta hai

👉 Matlab:

- **Editor ko storage ke behavior ka idea hai**
- **Document ko rendering ka idea hai**

✗ **Knowledge leak ho rahi hai**

✗ **Responsibilities thodi mix hain**

✗ **Isliye:**

Better design **working hai**, but **loosely coupled nahi hai**

● Optimal Design

Optimal design me **class sirf apna kaam jaanti hai**, baaki sab ka kaam **delegate** kar deti hai.

Same scenario but clean way:

- **Document:**
 - sirf elements store karta hai
 - render ka concept bhi nahi jaanta
- **DocumentEditor:**
 - sirf coordinator hai
 - usse nahi pata:
 - file me save ho raha hai
 - DB me save ho raha hai
 - kaise render ho raha hai

👉 Matlab:

- **No extra knowledge**
- **No responsibility leakage**
- **True abstraction**

✓ **Loose coupling**

✓ **Pure SRP**

✓ **Future-proof design**

🎯 One-Line Final Difference (Interview Gold)

Better design reduces problems, but optimal design removes unnecessary knowledge between classes.

Agar ek aur line bolni ho:

In better design, classes still know “how”; in optimal design, they only know “what”.

Tumhari thinking bilkul sahi direction me ja rahi hai bhaiya 🧡 Ye wahi observation hai jo **LLD interview me candidate ko stand-out** karwati hai.